Multi-Agent Reinforcement Learning With Spatial Structure Awareness for Topological Map-Based Path-Finding

Anzhe Wang and Zhiyun Lin, *Fellow, IEEE*

***Abstract*—Efficient Multi-Agent Path Finding (MAPF) is pivotal for warehouse logistics. While existing learning-based methods primarily rely on computationally intensive grid-based representations, topological maps offer a more flexible and scalable alternative - though this approach remains understudied. To address this gap, we propose a novel Multi-Agent Reinforcement Learning (MARL) framework for topological MAPF with three key innovations: (1) a graph-structured POMDP formulation utilizing our Breadth-First Neighbor-Limited Search (BFNLS) algorithm to define scalable observation/action spaces while maintaining fixed dimension; (2) a Graph Structure Awareness (GSA) model that combines spectral (eigenvalue-based) and spatial (graph convolutional network-based) analysis to integrate local subgraph features with global topological importance metrics; and (3) a cooperative MARL architecture employing Value Decomposition Networks (VDN) to explicitly model agent dependencies through graph-aware credit assignment. Simulation results show our method achieves superior success rates compared to baseline methods and planning efficiency than search-based methods, and the real-robot experiments show the effectiveness in a physical setting.**

***Index Terms*—Path planning for multiple mobile robots or agents, multi-robot systems.**

I. Introduction

With the development of multi-agent systems such as indoor service robots and factory robots, studies on multi-agent path planning problems have received increased attention [1], [2], [3]. Multi-agent path planning addresses the challenge of enabling multiple agents to find a conflict-free path from the start to the goal while minimizing the distance or time of travel as much as possible.

Recent advances in artificial intelligence have shifted MAPF research toward learning-based methods due to their superior generalization across varying agent numbers and tasks [4], [5], [6], [7]. Current approaches primarily employ grid-based representations, where agents perceive their environment through matrix observations and navigate grid cells. This framework enables adaptable path planning in diverse scenarios.

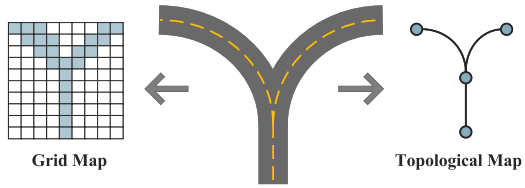


Fig. 1. Demonstration of topological maps v.s. grid maps: Topological maps represent spatial structures using fewer components while providing a more precise depiction.

However, in some scenarios, especially indoor scenarios like warehouses, topological maps with graph data are often more effective for MAPF tasks than grid maps with structured data. The graph enhances the efficiency of spatial planning and execution for the following reasons: **Spatial Compression:** Topological maps simplify the connectivity between regions and offer superior spatial compression, significantly reducing both the computational and the storage resources for planning. **Complex Structures Representation:** These maps are capable of representing more complex spatial configurations, allowing vertices to connect via edges at any angle, and the edges can be either straight lines or arbitrary curves, which enhances their capability in addressing complex path planning challenges. The benefits of topological maps are demonstrated in Fig. 1.

To unlock this efficiency for complex, high-agent-count scenarios, a scalable learning-based approach is needed. We formulate the topological MAPF problem as a POMDP and employ value-based MARL for training. However, applying learning-based methods to MAPF on topological maps introduces several significant challenges. First, unlike grid-based MAPF—where rectangular fields of view provide a natural input structure—the high variability of connectivity on topological maps complicates the definition of observation and action spaces. Second, designing observation mechanisms that simultaneously capture sufficient local and global information under partial observability is non-trivial. Third, developing neural network architectures and training frameworks capable of learning cooperative

Received 5 June 2025; accepted 29 October 2025. Date of publication 21 November 2025; date of current version 26 November 2025. This article was recommended for publication by Associate Editor K. Baxevani and Editor M. A. Hsieh upon evaluation of the reviewers' comments. This work was supported in part by the National Natural Science Foundation of China under Grant U23A20325 and Grant 62173118, in part by Guangdong Science and Technology Program under Grant 2024B1212010002, and in part by Shenzhen Science and Technology Program under Grant KQTD20221101093557010. *(Corresponding author: Zhiyun Lin.)*

The authors are with the School of Automation and Intelligent Manufacturing, Southern University of Science and Technology, Shenzhen 518055, China, and also with the Guangdong Provincial Key Laboratory of Fully Actuated System Control Theory and Technology, Shenzhen 518055, China (e-mail: linzy@sustech.edu.cn).

Digital Object Identifier 10.1109/LRA.2025.3635371

2377-3766 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: VIT-Amaravathi campus. Downloaded on October 31,2026 at 16:49:27 UTC from IEEE Xplore. Restrictions apply.

strategies—while effectively integrating graph structure and feature information—remains a critical challenge. To address these challenges, we propose a novel MARL framework for topological MAPF integrating graph-based representations. Firstly, we propose a novel observation mechanism for topological MAPF that integrates both local and global information. We propose Breadth-First Neighbor-Limited Search to define observations and actions with a neighbor-limited tree. This approach effectively leverages the graph structure while maintaining consistent observations and actions. Secondly, we design an information-rich observation representation for topological MAPF that effectively incorporates subgraph structure, local features, and global features. While existing methods often overlook global structural information, our method integrates local subgraph topology and real-time agent positions with global static topological features that indicate the strategical importance of vertices. Finally, we propose a Graph Structure Awareness (GSA) model specifically designed for topology-based MAPF problems. The model performs dual-channel analysis of observation information in both spectral and spatial domains through graph eigenvalue analysis and graph convolutional networks (GCNs), enabling accurate state-action value prediction. We then integrate GSA into a cooperative MARL framework that captures collaborative relationships via value decomposition, thereby improving state-action value estimation and enhancing multi-agent coordination.

Simulation results demonstrate that our model outperforms baseline models across various map sizes, and that our approach offers superior computational efficiency compared to search-based MAPF methods on topological maps. Furthermore, we provide real-world validation of our method's effectiveness through physical robot experiments.

II. Related Works

A. Search-Based MAPF

Search-based methods represent a fundamental approach to MAPF, where algorithms systematically explore solution spaces to find solution paths [8]. Conflict-Based Search (CBS) addresses this limitation through a two-level architecture: a high-level constraint tree resolves conflicts while low-level planners generate individual paths [9]. Recent variants like Explicit Estimation CBS (EECBS) incorporate online learning and inadmissible heuristics to achieve faster planning with bounded suboptimality [10]. Alternative approaches include Priority-Based Search (PBS) which improves solution quality through conflict-driven priority ordering [11]. ODrM$*$ combines conflict decomposition and operator decomposition to efficiently resolve conflicts by partitioning them into subsets [12].

Notably, several recent studies have adapted classical search-based algorithms such as CBS and PBS for multi-agent path finding on topological maps [13], [14]. Other approaches leverage topological braids to classify and reason about agent movement patterns during dynamic interactions [15]. While these methods offer valuable insights, they predominantly rely on centralized planning paradigms, which limit their applicability in decentralized and partially observable settings. In contrast, our work introduces a decentralized multi-agent reinforcement learning framework explicitly designed for topological maps with irregular connectivity, enabling scalable and adaptive decision-making under local observations.

B. Learning-Based MAPF

Recent advances in deep learning have spurred growing interest in learning-based MAPF with partially observable scenarios. An intuitive approach for MAPF is Independent Q-Learning (IQL) [6], where agents treat others as part of the environment. Communication mechanisms are often necessary to enhance cooperation and convergence [16]. A more widely used strategy is Centralized Training with Decentralized Execution, which leverages global information during training but allows decentralized action based on local observations [4]. Some studies combine reinforcement and imitation learning to ease training challenges [4], [17], [18], while others incorporate heuristic or global guidance for complex environments [6], [7], [19].

Current learning-based MAPF methods remain limited to grid-based representations, failing to leverage: (1) the natural efficiency of topological maps, and (2) the critical role of global structural features.

III. Problem Formulation

A. MAPF Formulation

In a MAPF problem, a map is defined in an undirected graph $G = (V, E)$, which is pre-constructed from the physical environment. The adjacency matrix A for graph G is defined as:

$$A(i,j) = \begin{cases} 1 & \text{if } (v_i, v_j) \in E, \\ 0 & \text{otherwise.} \end{cases} \tag{1}$$

The scenario involves N agents, each with a start vertex $v_s \in V$ and goal vertex $v_g \in V$. The model computes conflict-free paths in discrete time steps, during which each agent can either move to an adjacent vertex or stay at its current vertex. Two types of conflicts are prohibited: (1) vertex conflicts, where multiple agents occupy the same vertex simultaneously, and (2) edge conflicts, where agents traverse the same edge in opposite directions. This formulation implicitly assumes that agents traversing any two distinct, non-conflicting edges do not physically collide, and the topological maps in our work are designed to be consistent with this assumption.

B. Partially Observable Environments

We model the partially observable MAPF problem as a POMDP. Formally, the POMDP is defined as a 7-tuple $\langle \mathcal{S}, \mathcal{A}, \tau, R, \Omega, O, \gamma \rangle$ where:

- $\mathcal{S}$ is the set of states.
- $\mathcal{A} = \prod_{i=1}^{N} \mathcal{A}_i$ is the joint action space and $\mathcal{A}_i$ is the set of actions for agent i.
- $\tau : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to [0, 1]$ is the conditional transition probabilities $P(s'|a, s)$.
- $R : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ is the reward function.
- $\Omega = \prod_{i=1}^{N} \Omega_i$ is the joint observation space and Ω_i is the observation space for agent i.

Authorized licensed use limited to: VIT-Amaravathi campus. Downloaded on October 31,2026 at 16:49:27 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: Breadth-First Neighbor-Limited Search (BFNLS).

Input: Graph $G = (V, E)$, root vertex $v_r \in V$, maximum number of neighbors k
Output: Observable vertex set V^{v_r} and parent-child edge set E^{v_r}

```
Q ← new Queue()
Enqueue v_r to Q
V^{v_r} ← {v_r}, E^{v_r} ← ∅
while Q is not empty and |V^{v_r}| < k do
  v_p ← Dequeue from Q
  for each neighbor v_c of v_p in G do
    if |V^{v_r}| ≥ k then
      break
    end if
    if v_c ∉ V^{v_r} then
      Enqueue v_c to Q
      V^{v_r} ← V^{v_r} ∪ {v_c}, E^{v_r} ← E^{v_r} ∪ {(v_p, v_c)}
    end if
  end for
end while
return V^{v_r}, E^{v_r}
```

- $O : \Omega \times \mathcal{S} \times \mathcal{A} \to [0, 1]$ is the conditional observation function $P(o|s, a)$.
- $\gamma \in [0, 1)$ is the discount factor.

In our environment, the observation and state transitions are deterministic, and the transitions and observations of an agent depend on its current state and action.

IV. Method

This section presents our approach to the partially observable topological MAPF. We first define the observation space, which consists of an observation range and observation information. The action space specifies discrete navigational choices, while the reward function balances path efficiency and cooperation. Next, we present our spatial structure awareness model, a novel framework designed to explicitly capture and utilize the topological properties of the environment for MAPF. Finally, we detail our training and execution framework, where agents learn cooperative policies through MARL.

A. Limited Observation for Topological MAPF

The observation is defined as a local topological subgraph centered around its current position. This local view is dynamically constructed based on the agent's vertex location within the graph, aiming to capture the local topological structure and environmental information for path-finding.

The observation is composed of two key components: the observation range and the observation information. The observation range defines the set of local vertices and edges an agent can perceive from its current position in the environment. In contrast, the observation information refers to the specific features and data associated with those components within the range. In other words, the range determines which vertices and edges are visible, while the observation information describes the properties of those components that the agent uses for decision-making.

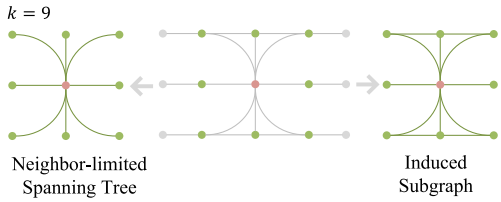


Fig. 2. Neighbor-limited spanning tree and the induced subgraph within the observation range.

We introduce Breadth-First Neighbor-Limited Search (BFNLS) to select a fixed number of neighboring vertices. Different from k-limited BFS, which limits the search scope by a predefined depth, BFNLS adaptively defines the observation range for an agent at vertex v_r by constructing a neighbor-limited tree, whose total number of vertices is a predefined parameter k, as illustrated in the left part of Fig. 2. As detailed in Algorithm 1, the BFNLS algorithm constructs a neighbor-limited tree rooted at v_r containing exactly k vertices.

BFNLS induces a tree structure capturing the agent's observable neighborhood. We formally define this structure:

Definition 1 (Neighbor-Limited Spanning Tree): Given a root vertex $v_r \in V$ and a predefined parameter k, the neighbor-limited spanning tree $T^{v_r} = (V^{v_r}, E^{v_r})$ is constructed as follows:

- **Vertex Set**: The vertex set V^{v_r} consists of v_r and all vertices reachable from v_r via BFNLS, ensuring $|V^{v_r}| = k$.
- **Edge Set**: The edge set E^{v_r} encodes parent-child relationships, where each edge $(v_p, v_c) \in E^{v_r}$ indicates v_c is discovered through BFNLS expansion from v_p.

For any vertex $v_t \in V^{v_r}$ in a neighbor-limited spanning tree T^{v_r}, the path $P_{v_r \to v_t}$ from the root vertex v_r to v_t can be uniquely determined by recursively following parent-child relationships. That is, starting from v_t, we trace backward through each parent vertex using the edge set E^{v_r} until reaching the root v_r. The complete path is then formed by collecting these vertices in reverse order, beginning with v_r followed by each subsequent child vertex along the chain.

The neighbor-limited spanning tree defines the scope of observable vertices, so we introduce the tree-induced subgraph to represent the local structure, as shown in the right part of Fig. 2. The induced subgraph is denoted as:

$$G^{v_r} = (V^{v_r}, \bar{E}^{v_r}), \tag{2}$$

where $\bar{E}^{v_r} \subseteq E$ is the set of edges between vertices in V^{v_r}:

$$\bar{E}^{v_r} = \{(u, v) \in E \mid u \in V^{v_r}, v \in V^{v_r}\}. \tag{3}$$

Remark 1: Our observation range improves the spatial awareness and collaboration, especially in the two scenarios.

Narrow Corridor: In narrow corridors, agents need to see farther ahead. The fixed observation includes enough forward vertices to spot approaching agents while keeping the input manageable.

Authorized licensed use limited to: VIT-Amaravathi campus. Downloaded on October 31,2026 at 16:49:27 UTC from IEEE Xplore. Restrictions apply.

Intersection: At intersections, agents need to watch all directions. The fixed observation covers key approaching paths from all sides without being overwhelmed by too much information.

B. Observation Information Design

In the previous subsection, we utilize a neighbor-limited spanning tree to define the observation range and its induced subgraph. In this subsection, we define the observation information within the observation range and finally formalize the observation space.

For an agent at vertex v_r, its observation information is within the range of the subgraph $G^{v_r} = (V^{v_r}, \bar{E}^{v_r})$ and is composed of the subgraph structure and the vertices' features.

Subgraph structure: The subgraph structure within a local observation range serves as crucial reference information for decision-making. In computational representations, we employ adjacency matrices of size $k \times k$ to represent the subgraph, and the elements of the matrices indicate whether pairs of vertices are adjacent or not in the subgraph. The adjacency matrix $A^{v_r} \in \mathbb{R}^{k\times k}$ records the subgraph structure information of subgraph G^{v_r}. The (i, j)-element of A^{v_r} is defined as follows:

$$A^{v_r}(i,j) = \begin{cases} 1 & \text{if } (v_i, v_j) \in \bar{E}^{v_r}, \\ 0 & \text{otherwise.} \end{cases} \tag{4}$$

Vertices' features: The vertices' features include the information related to the vertices within the observation range. In our work, we utilize information at different scopes, including both local and global levels [7].

1) **Local information**: It encompasses other agents' positions within the observation range, which is critical for making effective collision-avoidance decisions.
2) **Global information**: It encompasses both heuristic guidance and structural graph-theoretic features, including graph path-diversity, and betweenness centrality (identifying critical routing vertices). These elements are static but insightful information with global guidance, enabling agents to balance heuristic-driven navigation with structural awareness optimization under partial observability constraints.

We use the vertices' feature vectors to record these features for k observable vertices in V^{v_r}. Without loss of generality, we denote the k vertices in V^{v_r} as $\bar{v}_1, \bar{v}_2, \ldots, \bar{v}_k$, and let $\bar{v}_1 = v_r$. These vectors are assembled into a feature matrix $X^{v_r} \in \mathbb{R}^{k\times 5}$:

$$X^{v_r} = \begin{bmatrix} x_{\bar{v}_1}^T \\ x_{\bar{v}_2}^T \\ \vdots \\ x_{\bar{v}_k}^T \end{bmatrix}, \tag{5}$$

where each $x_{\bar{v}_1}, x_{\bar{v}_2}, \ldots, x_{\bar{v}_k}$ is a 5-dimensional column vector.

We now explain the features used in our approach, which are agent occupancy, average neighbor degree, heuristic information, path diversity, and betweenness centrality. The feature vector $x_{\bar{v}_i}^T$ is denoted as $x_{\bar{v}_i}^T = [x_O(i), x_N(i), x_H(i), x_C(i), x_B(i)]$ including local features $x_O(i)$, $x_N(i)$, as well as global features $x_H(i)$, $x_C(i)$, and $x_B(i)$ for vertex $\bar{v}_i \in V^{v_r}$.

Firstly, we provide precise descriptions for local features.

- Occupancy $x_O(i)$: It records the occupancy of some agent at vertex $\bar{v}_i$:

$$x_O(i) = \begin{cases} 1 & \text{if occupied by an agent,} \\ 0 & \text{otherwise.} \end{cases} \tag{6}$$

- Average Neighbor Degree $x_N(i)$: It quantifies local network density by measuring the mean degree of adjacent vertices. The *neighbor set* of vertex v_i is $N_i \subseteq V$, consisting of all vertices directly connected to v_i, i.e., $N(v_i) = \{v_j \in V \mid (v_i, v_j) \in E\}$, and $|N(v_i)| = \deg(v_i)$ represents the vertex degree. The average neighbor degree for vertex $\bar{v}_i$ is:

$$x_N(i) = \frac{1}{\deg(\bar{v}_i)} \sum_{v_j \in N(\bar{v}_i)} \deg(v_j). \tag{7}$$

Secondly, we provide precise descriptions for global features.

- Heuristic Distance $x_H(i)$: Heuristic distance can guide agents in learning effective strategies [6], [19]. The heuristic distance is the shortest path distance d_i from $\bar{v}_i$ to the goal vertex subtracting the shortest path distance d_r from v_r to the goal vertex, i.e.,

$$x_H(i) = d_i - d_r. \tag{8}$$

- Path Diversity $x_C(i)$: Path diversity captures the diversity of path choices to the goal by considering all possible paths, regardless of length. It is computed using the matrix exponential of the adjacency matrix A of graph G, yielding C:

$$C = \sum_{m=0}^{\infty} \frac{A^m}{m!}, \tag{9}$$

where A^m denotes the m-th power of the adjacency matrix A, which corresponds to the number of paths of length m between vertices.

From the matrix C, we utilize two entries: $C_{\bar{v}_i, v_g}$ (between current vertex $\bar{v}_i$ and goal v_g) and C_{v_r, v_g} (between root vertex v_r and v_g). To prevent numerical range disparities, we define a relative path diversity:

$$x_D(i) = \log \frac{C_{\bar{v}_i, v_g}}{C_{v_r, v_g}}. \tag{10}$$

- Betweenness Centrality $x_B(i)$: Betweenness centrality evaluates vertex congestion risks through shortest-path throughput analysis. For vertex $\bar{v}_i$, it is computed as:

$$x_B(i) = \sum_{v_s, v_t \in V} \frac{\sigma(v_s, v_t | \bar{v}_i)}{\sigma(v_s, v_t)}, \tag{11}$$

where $v_s, v_t \in V$ are any start vertex and end vertex in graph G, and $v_s \neq \bar{v}_i$, $v_s \neq v_t$, $v_t \neq \bar{v}_i$. $\sigma(v_s, v_t)$ is the total number of shortest paths from vertex v_s to vertex v_t within the graph G, and $\sigma(v_s, v_t | \bar{v}_i)$ is the count of those paths that pass through vertex $\bar{v}_i$.

Authorized licensed use limited to: VIT-Amaravathi campus. Downloaded on October 31,2026 at 16:49:27 UTC from IEEE Xplore. Restrictions apply.

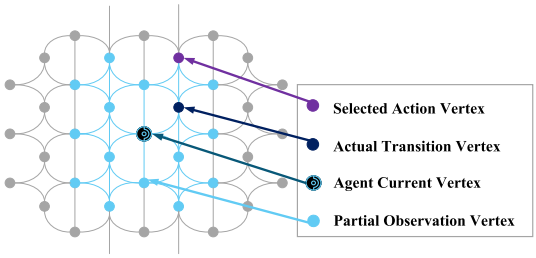


Fig. 3. Demonstration of selected action and its corresponding movement in execution.

After introducing the features, we obtain the feature matrix X^{v_r} at vertex v_r, that is,

$$X^{v_r} = \begin{pmatrix} x_O(1) & x_N(1) & x_H(1) & x_D(1) & x_B(1) \\ x_O(2) & x_N(2) & x_H(2) & x_D(2) & x_B(2) \\ x_O(3) & x_N(3) & x_H(3) & x_D(3) & x_B(3) \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ x_O(k) & x_N(k) & x_H(k) & x_D(k) & x_B(k) \end{pmatrix}. \quad (12)$$

In conclusion, for an agent at any vertex $v_r \in V$, its observation o^{v_r} is constructed by encoding the adjacency matrix A^{v_r} of the subgraph G^{v_r} and the feature matrix X^{v_r}, namely,

$$o^{v_r} = (A^{v_r}, X^{v_r}). \quad (13)$$

The observation space Ω_i for each agent i is defined as the set of all possible observations generated at every vertex in the graph:

$$\Omega_i = \{o^{v_r} \mid v_r \in V\}. \quad (14)$$

The joint observation space is defined as: $\Omega = \Pi_{i=1}^{N} \Omega_i$.

C. Action Space for Topological MAPF

The action space is based on a neighbor-limited spanning tree similar to the design of observation. The dimension of the action space is k, which corresponds to the number of observable vertices in V^{v_r}. For an agent at vertex $v_r \in V$, the action space can be defined as a set of indices:

$$\mathcal{A} = \{1, 2, \ldots, k\}. \quad (15)$$

Each action represented by an index i in the action space corresponds to choosing to move toward the vertex $v_i \in V^{v_r}$.

In MAPF, agents in execution can only move to adjacent vertices or wait. Action 1 in $\mathcal{A}$ is functionally equivalent to the wait action, which corresponds to remaining at the current vertex. Moreover, the action space $\mathcal{A}$ defined in (15) includes transitions to potentially non-adjacent vertices. During execution, if the agent selects an action $a \in \mathcal{A}$ that targets a non-adjacent vertex v', it means the agent only takes one-step transition from current vertex v_r to the next adjacent vertex v_s along the shortest path $P_{v_r \to v'}$. This ensures that the agent's motion adheres to the environment's connectivity constraints while preserving the semantics of the original action space $\mathcal{A}$. Fig. 3 shows the process: An agent chooses an action, moving towards the vertex, but it only moves one step to the adjacent vertex along the shortest path in execution.

TABLE I
REWARD TABLE FOR AGENT ACTIONS

State, Action	Reward
Moving toward the goal	0.0
Moving away from the goal	-0.2
Occurrence of a conflict	-1.0
Reaching the goal	10.0
Toggle between vertices	-0.4

D. Reward Design

The reward design for our topological map based MAPF is shown in Table I.

Moving toward the goal means that an agent's movement to a new vertex reduces the shortest path distance to the goal, while moving away increases this distance. To avoid conflicts, there is a negative reward for actions that cause vertex conflict and edge conflict. A positive reward is given to the agent if it reaches goal vertex. To discourage oscillatory movements between vertices, a negative reward is given to penalize such behavior.

E. Graph Structure Awareness Model

We propose a model called the graph structure awareness (GSA) model to process graph-based information and predict the state-action value for diverse scenarios. For an agent at a vertex v_r, its observation $o^{v_r} = (A^{v_r}, X^{v_r})$, includes the adjacency matrix A^{v_r} and the feature matrix X^{v_r}. The model first processes observation, then performs feature extraction based on eigenvalues analysis and a graph neural network, and finally outputs predicted state-action values.

In the observation processing stage, we construct two subgraphs to reflect the impact of other agents' occupancy based on the subgraph structure and the agent's occupancy in the observation. Towards this purpose, we denote $V_{occ}^{v_r} \subseteq V^{v_r}$ the set of vertices that are occupied by agents at current time. The two subgraphs are defined as:

- Static graph: $G^{v_r} = (V^{v_r}, \bar{E}^{v_r})$, as described in (2).
- Dynamic graph: $G_{free}^{v_r} = (V^{v_r}, \bar{E}^{v_r} \setminus E_{occ}^{v_r})$ is the one by removing edges connected to $V_{occ}^{v_r}$, where:

$$E_{occ}^{v_r} = \{(u, v) \in E \mid u \in V_{occ}^{v_r} \text{or} v \in V_{occ}^{v_r}\}. \quad (16)$$

In the feature extraction stage, to analyze both the intrinsic graph characteristics and the partial observation features, a dual-path approach that combines spectral analysis with graph convolution is proposed, as shown in Fig. 4. Specifically, we leverage the two subgraphs G^{v_r} and $G_{free}^{v_r}$ to learn feature representations: While eigenvalues reveal the spectral properties, graph convolution operates on structures to aggregate neighborhood information.

Firstly, our model quantifies the intrinsic structure of subgraphs through spectral analysis, solving for the eigenvalues of both G^{v_r} and $G_{free}^{v_r}$. Then, the following procedure is applied:

1) Construct adjacency matrices $(A^{v_r}, A_{free}^{v_r})$ and degree matrices $(D^{v_r} = \text{diag}(\sum_j A^{v_r}(i,j)), D_{free}^{v_r} = \text{diag}(\sum_j A_{free}^{v_r}(i,j)))$.

Authorized licensed use limited to: VIT-Amaravathi campus. Downloaded on October 31,2026 at 16:49:27 UTC from IEEE Xplore. Restrictions apply.

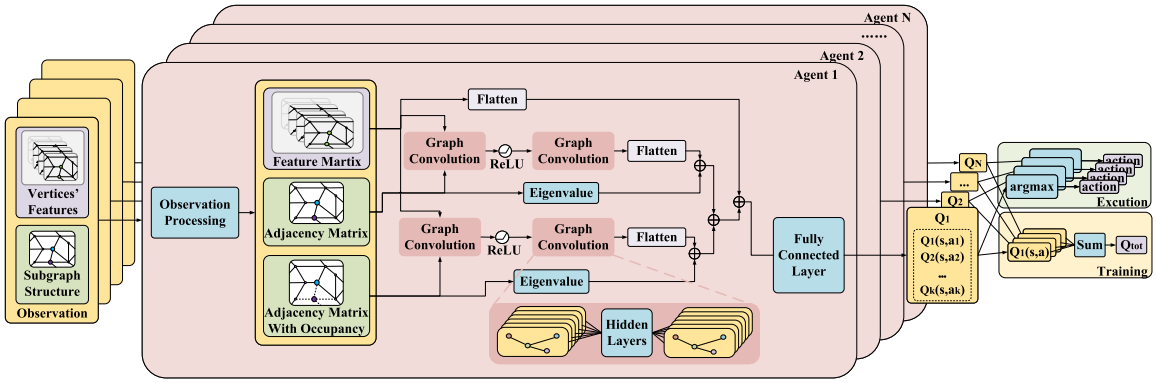


Fig. 4. Training and execution workflow with graph structure awareness model.

2) Derive Laplacians $L^{v_r} = D^{v_r} - A^{v_r}$ and $L^{v_r}_{free} = D^{v_r}_{free} - A^{v_r}_{free}$.
3) Compute the Laplacian eigenvalues (λ^{v_r}, $\lambda^{v_r}_{free}$) for both subgraphs.

Secondly, our model employs graph convolution to learn feature representations from two subgraphs: G^{v_r} and $G^{v_r}_{free}$. The input feature matrix $H^{(0)}$ are taken directly from the observation, i.e, $H^{(0)} = X^{v_r}$. Taking G^{v_r} as an example, the graph convolutional module operates on input features $H^{(0)}$ with learnable parameters $W^{(l)}$ at layer l, where the propagation rule is given by:

$$H^{(l+1)} = \mathrm{ReLU}(D^{v_r-\frac{1}{2}} A^{v_r} D^{v_r-\frac{1}{2}} H^{(l)} W^{(l)}). \tag{17}$$

As shown in Fig. 4, the model processes vertex features through two parallel graph convolution paths. One path inputs A^{v_r} with $H^{(0)}$, and another inputs $A^{v_r}_{free}$ with $H^{(0)}$. Each path applies independent graph convolution layers and feature flattening. The flattened features are concatenated with both adjacency matrices' eigenvalues and vertex features. This combined vector passes through fully-connected layers before the output layer generates state-action values.

F. Cooperative Multi-Agent Reinforcement Learning

To address the challenge of coordinating multiple agents, our work adopts the Centralized Training with Decentralized Execution framework. While decentralized approaches offer scalability, they risk inter-agent conflicts due to a lack of explicit coordination. Our method uses a Value Decomposition Network (VDN) [20] to learn a joint action-value function, $Q_{tot}(o, a; \theta)$, which is decomposed as the sum of individual agent value functions. This decomposition enables agents to execute actions based only on their local observations while learning cooperative behaviors during a centralized training phase. Crucially, VDN ensures that maximizing an individual agent's action-value is consistent with maximizing the global action-value, promoting effective multi-agent coordination.

In the training stage, we maintain a training network $Q_i(o_i, a_i; \theta)$ and a target network $Q'_i(o_i, a_i; \theta')$ with the same GSA model structure. The total Q-value is defined as $Q_{tot}(o, a; \theta) = \sum_{i=1}^{N} Q_i(o_i, a_i; \theta)$, where $o = (o_1, \ldots, o_N)$ and $a = (a_1, \ldots, a_N)$ is the joint observation and action, and θ denotes the parameters of the training network. The joint reward for each agent is $r = (r_1, \ldots, r_N)$. Similarly, the target network computes $Q'_{tot}(o, a; \theta') = \sum_{i=1}^{N} Q'_i(o_i, a_i; \theta')$, with parameters θ'. The training network is optimized end-to-end by minimizing the loss:

$$L(\theta) = \frac{1}{B} \sum_{j=1}^{B} \left(y^{(j)} - Q_{tot}(o, a; \theta) \right)^2, \tag{18}$$

where B is the batch size, and the target value is: $y^{(j)} = r^{(j)} + \gamma \max_{a'} Q'_{tot}(o', a'; \theta')$, with $r^{(j)} = \sum_{i=1}^{N} r_i^{(j)}$ as the sum of joint reward, γ as the discount factor, o' as the next joint observation, and a' as the next joint action. The target network parameters θ' are periodically updated by copying training network parameters θ to ensure training stability.

In the execution stage, each agent i observes o_i and selects the optimal action from the trained Q-network with parameters θ_i, i.e., $a_i = \arg\max_{a_i} Q_i(o_i, a_i; \theta_i)$. The decentralized execution of all agents, guided by the trained parameters $\theta = (\theta_1, \ldots, \theta_N)$, ensures that the joint action maximizes $Q_{tot}(o, a; \theta)$, achieving effective inter-agent coordination.

V. Simulation and Experiment

For evaluation, we conduct two sets of simulations and a real-robot experiment. First, we test the scalability of our method on topological maps of varying sizes against baseline models. Second, we compare the performance and show the advantages of our method with search-based methods. Finally, we validate our method's practical applicability with a real-robot experiment.

A. Implement Details

Our method is implemented on a workstation equipped with an Intel Core i7-13700 CPU, an NVIDIA GeForce RTX 4080 GPU, and 64 GB of RAM. The GSA model employs a two-layer Graph Convolutional Module encoder with a hidden dimension of 32 and ReLU activation functions. The encoder's output, fused with spectral graph features, is fed into a four-layer MLP head with hidden layer dimensions of 512, 256, 256, and 128.

Authorized licensed use limited to: VIT-Amaravathi campus. Downloaded on October 31,2026 at 16:49:27 UTC from IEEE Xplore. Restrictions apply.

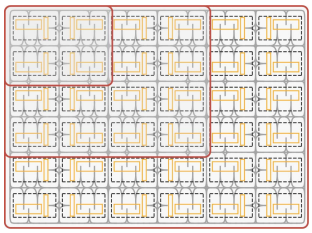


Fig. 5. Demonstration of different sizes of topological maps.

For training, we utilize the Adam optimizer with a learning rate of 1×10^{-4}, a batch size of 16, a discount factor γ of 0.99 and target update frequency of every 50 episodes. All agents share identical network parameters. An Epsilon-Greedy exploration strategy is adopted, with epsilon decayed linearly from 1.0 to 0.01. The training was set to run for a fixed episodes number. For the MARL training, the use of the VDN under the CTDE framework provided good training stability.

B. Model Performance Comparison on Topological Maps

We implement our GSA model in a warehousing scenario for MAPF tasks to evaluate its performance on a topological map. To evaluate the performance of our model, we use two different models for comparison.

1) MLP Baseline: This model operates without any graph-structural prior knowledge, serving to evaluate the advantage of our graph-aware architecture over a generic closed box model. It consists of four fully-connected layers with hidden dimensions of 512, 256, 256, and 128, and uses ReLU activation functions.

2) Spectral GCN Baseline: This baseline is introduced to assess the benefits of our model's dual-graph design and hybrid spectral-spatial feature fusion mechanism. It comprises two graph convolutional layers with a hidden dimension of 32 and ReLU activation. The model receives the same input observation as our GSA model—namely, the subgraph adjacency matrix A^{v_r} and feature matrix X^{v_r}—and feeds into the same downstream fully-connected layers (with dimensions 512, 256, 256, 128) as used in our proposed framework.

The test maps consist of a series of generalized warehouse logistics maps (with areas of 2×2, 4×4, and 6×6, as demonstrated in Fig. 5) whose edges are bidirectional. We train policies using 8, 24, and 64 agents on each of the three maps and evaluate with different number of agents. In each training episode, the start and goal positions for all agents were randomly selected from the valid vertices on the map.

We compare the arrival rate (AR) success rate (SR) and the step costs (Cost). The arrival rate refers to the percentage of agents that reach goals: $\text{AR} = \frac{N_r}{N}$, where N_r is the number of agents that reach goals and N is the total number of agents in all scenarios. The success rate is defined as: $\text{SR} = \frac{N_s}{N_{all}}$, where N_s is the number of successful scenarios and N_{all} is the number of all scenarios. The AR and SR metric reflects the algorithm's ability to generate collision-free paths. AR and SR decrease if collisions prevent individual agents from reaching their goals. The cost denotes the maximum steps that the last agent reaches its goal (If an agent fails to reach its goal, the step cost is the maximum number of steps allocated for the task): $\text{Cost} = \max_{i=1}^{N}(\min(S_i, S_{max}))$, where N is the number of agents, S_i denotes the step cost for agent i to reach the goal, and S_{max} is the maximum number of steps. The results are presented in Fig. 6. The results indicate that our model exhibits better performance compared to the MLP without graph structure and a spectral GCN model.

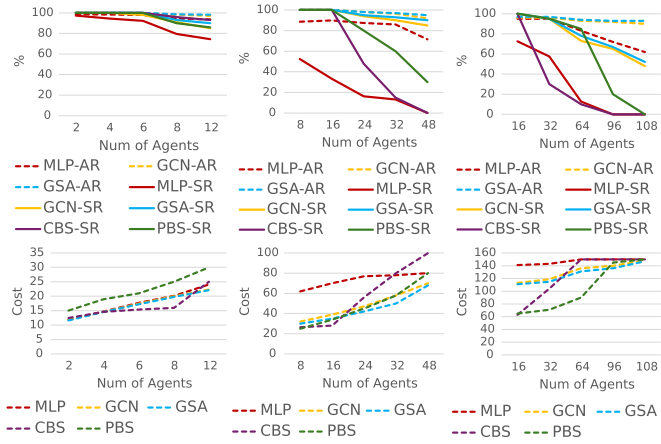


Fig. 6. Performance comparison across different numbers of agents and maps.

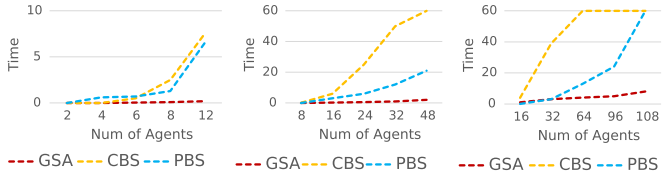


Fig. 7. Comparison of average computation time (in seconds).

To test our model's reliance on the vertex order from the BFNLS algorithm, we conduct an ablation study. We feed our trained model observations where the k vertices are randomly permuted. This change cause the AR to drop from 94.8% to nearly 0% in the 2x2 map scenario with 10 agents, confirming the model critically depends on the structured order generated by the breadth-first search.

C. Performance Comparison With Search-Based Methods

In this subsection, we compare our method against the classic search paradigm, specifically CBS and PBS, on topological map. We focus our comparison on three metrics: computation time required to find a solution, success rate and solution cost. The success rate and solution cost comparison are shown in Fig. 6 and the average computation time is shown in Fig. 7.

Our GSA model requires only a single, rapid forward pass for inference, resulting in a negligible and constant computation time. Consequently, it consistently finds a path within the time limit (60 s), maintaining a relatively high success rate across all agent numbers. In contrast, the planning time for search based methods grows with the number of agents and lead to a sharp drop in success rate as agent density increases.

Authorized licensed use limited to: VIT-Amaravathi campus. Downloaded on October 31,2026 at 16:49:27 UTC from IEEE Xplore. Restrictions apply.

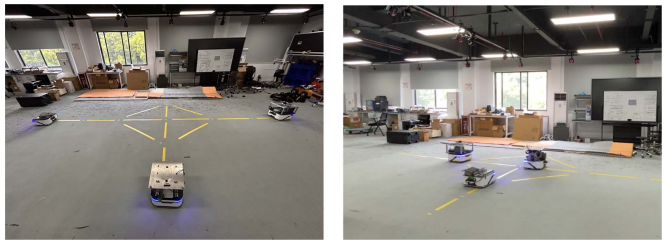


Fig. 8. Real robots experiment in an intersection.

When search-based methods successfully find a solution (i.e., for a low number of agents, given sufficient time), it yields the optimal or near optimal cost. However, this optimality comes at the expense of computational time and a low success rate in more complex scenarios. Our method, while not guaranteeing optimality, consistently and rapidly produces feasible solutions.

D. Real-Robot Experiment

To further validate the feasibility and effectiveness of our proposed MARL framework in the real world, we design and conduct a real robot experiment. We deploy three AgileX Tracer Mini mobile robots (MRs). The agents determine their corresponding vertex on the pre-defined topological map based on their physical location and execute the actions (wait or move to an adjacent vertex) dictated by the model's output to achieve autonomous collision avoidance and passage through the intersection as shown in Fig. 8. The results demonstrate that the three MRs successfully navigated the intersection without collision.

VI. Conclusion

In this letter, we propose a MAPF method based on partially observable topological maps, aimed at enhancing planning and operational efficiency. We formulate the topological MAPF as a POMDP and design observation and action spaces based on neighbor-limited spanning trees. Our graph structure awareness (GSA) model achieves accurate state-action value estimation by jointly analyzing local and global graph information through eigenvalue processing and graph convolutions, while the integrated value decomposition network (VDN) assesses individual agent contributions to the collective outcome, together enabling MAPF performance. Simulation results demonstrate that our GSA model outperforms baseline models across diverse topological maps and cost less computational time compared to search-based methods, while the real world experiment shows the effectiveness.

Future work will focus on two key directions. First, we will investigate integrating communication mechanisms to enhance agent collaboration on topological maps. Second, we plan to develop a framework capable of handling variable-sized observations by replacing the current flattening operation with a more robust, permutation-invariant graph pooling or attention-based readout mechanism. We believe this will resolve the model's current dependency on a fixed-size input and significantly enhance its adaptability and robustness in more diverse scenarios.

References

[1] H. Ma, "Graph-based multi-robot path finding and planning," *Curr. Robot. Rep.*, vol. 3, no. 3, pp. 77–84, 2022.

[2] F. Zhuang, T. Huang, Q. Xu, Y.-J. Gong, and J. Liu, "A robust lifelong multi-agent path finding with active conflict resolution and decentralized execution," *IEEE Robot. Autom. Lett.*, vol. 10, no. 5, pp. 4652–4659, May 2025.

[3] W. Li, Z. Ma, and Y. Yu, "Proactive multi-robot path planning via Monte Carlo congestion prediction in intralogistics," *IEEE Robot. Autom. Lett.*, vol. 10, no. 5, pp. 4588–4595, May 2025.

[4] G. Sartoretti et al., "PRIMAL: Pathfinding via reinforcement and imitation multi-agent learning," *IEEE Robot. Autom. Lett.*, vol. 4, no. 3, pp. 2378–2385, Jul. 2019.

[5] M. Damani, Z. Luo, E. Wenzel, and G. Sartoretti, "Primal_2: Pathfinding via reinforcement and imitation multi-agent learning-lifelong," *IEEE Robot. Autom. Lett.*, vol. 6, no. 2, pp. 2666–2673, Apr. 2021.

[6] Z. Ma, Y. Luo, and H. Ma, "Distributed heuristic multi-agent path finding with communication," in *Proc. 2021 IEEE Int. Conf. Robot. Automat.*, Xi'an, China, 2021, pp. 8699–8705.

[7] C. He, T. Yang, T. Duhan, Y. Wang, and G. Sartoretti, "ALPHA: Attention-based long-horizon pathfinding in highly-structured areas," in *Proc. 2024 IEEE Int. Conf. Robot. Automat.*, Yokohama, Japan, 2024, pp. 14576–14582.

[8] Y. Hua, Y. Wang, and Z. Ji, "Adaptive lifelong multi-agent path finding with multiple priorities," *IEEE Robot. Autom. Lett.*, vol. 9, no. 6, pp. 5607–5614, Jun. 2024.

[9] G. Sharon, R. Stern, A. Felner, and N. R. Sturtevant, "Conflict-based search for optimal multi-agent pathfinding," *Artif. Intell.*, vol. 219, pp. 40–66, 2015.

[10] J. Li, W. Ruml, and S. Koenig, "EECBS: A bounded-suboptimal search for multi-agent path finding," in *Proc. AAAI Conf. Artif. Intell.*, 2021, pp. 12353–12362.

[11] H. Ma, D. Harabor, P. J. Stuckey, J. Li, and S. Koenig, "Searching with consistent prioritization for multi-agent path finding," in *Proc. AAAI Conf. Artif. Intell.*, Honolulu, HI, USA, 2019, pp. 7643–7650.

[12] C. Ferner, G. Wagner, and H. Choset, "ODrM∗ optimal multirobot path planning in low dimensional search spaces," in *Proc. 2013 IEEE Int. Conf. Robot. Automat.*, Karlsruhe, Germany, 2013, pp. 3854–3859.

[13] S. Song, K.-I. Na, and W. Yu, "Corridor occupancy-based multi-agent pathfinding in topological maps," in *Proc. 13th Int. Conf. Inf. Commun. Technol. Convergence*, Irbid, Jordan, 2022, pp. 181–183.

[14] S. Song, K.-I. Na, and W. Yu, "Anytime lifelong multi-agent pathfinding in topological maps," *IEEE Access*, vol. 11, pp. 20365–20380, 2023.

[15] C. I. Mavrogiannis and R. A. Knepper, "Multi-agent path topology in support of socially competent navigation planning," *Int. J. Robot. Res.*, vol. 38, no. 2/3, pp. 338–356, 2019.

[16] Z. Ma, Y. Luo, and J. Pan, "Learning selective communication for multi-agent path finding," *IEEE Robot. Autom. Lett.*, vol. 7, no. 2, pp. 1455–1462, Apr. 2022.

[17] J. Xie et al., "Crowd perception communication-based multi- agent path finding with imitation learning," *IEEE Robot. Autom. Lett.*, vol. 9, no. 10, pp. 8929–8936, Oct. 2024.

[18] Y. Wang, B. Xiang, S. Huang, and G. Sartoretti, "SCRIMP: Scalable communication for reinforcement-and imitation-learning-based multi-agent pathfinding," in *Proc. 2023 IEEE/RSJ Int. Conf. Intell. Robots Syst.*, Detroit, MI, USA, 2023, pp. 9301–9308.

[19] Q. Lin and H. Ma, "SACHA: Soft actor-critic with heuristic-based attention for partially observable multi-agent path finding," *IEEE Robot. Autom. Lett.*, vol. 8, no. 8, pp. 5100–5107, Aug. 2023.

[20] P. Sunehag et al., "Value-decomposition networks for cooperative multi-agent learning," 2017, arXiv:1706.05296.

Authorized licensed use limited to: VIT-Amaravathi campus. Downloaded on October 31,2026 at 16:49:27 UTC from IEEE Xplore. Restrictions apply.